SAT-Erfüllbarkeit von booleschen Schaltkreisen und die Konsequenzen

Analyse, Reduktionen, Implementierung (subgraph-sat-solver) und Entwurfsprinzipien

Stephan Epp

M.Sc.

12. Oktober 2026

Inhaltsverzeichnis

1 Einführung **3**

2 Begriffe und Modellierung **3**
2.1 Boolesche Schaltkreise als Graphen und Formeln . . . 3
2.2 Graph-Modelle: Clique und Subgraph . . . 4

3 Formale Reduktionskette und Beweise **4**
3.1 Satz: 3-SAT auf Clique (klassische Konstruktion) . . . 4
3.2 Korollar: Clique → Subgraph (K_k als Muster) . . . 4
3.3 Implizite Korrektheitsforderungen für die Subgraph-Oracle-Schnittstelle . . . 5

4 Komplexitätstheoretische Konsequenzen **5**
4.1 NP-Vollständigkeit und Folgen . . . 5
4.2 Auswirkungen auf höhere Klassen . . . 5
4.3 Parameterized Complexity . . . 5

5 Implementierung (subgraph-sat-solver) **6**

6 SAT-Erfüllbarkeit für Schaltkreise (ISCAS-85/89) **7**
6.1 Vorgehen . . . 7
6.2 Typische Befunde (generische Prinzipien) . . . 7
6.3 Empfehlungen für die Praxis mit ISCAS . . . 8

7 Design- und Entwurfsprinzipien für SAT-freundliche Schaltkreise **8**
7.1 Prinzipien . . . 8

8 Beweise wichtiger Aussagen **8**

9 Praktische Bewertung, Limitierungen und wirtschaftliche Relevanz **9**
9.1 Leistung und Skalierbarkeit . . . 9
9.2 Wirtschaftliche Bedeutung . . . 9

10 Empfehlungen zur Nutzung **9**

11 Schluss **10**
11.1 Reduktion und Vorverarbeitung gezielt verbessern 10
11.2 Exakte und heuristische Verfahren sinnvoll kombinieren 10
11.3 Belege und Korrektheit durchgängig absichern 11
11.4 Benchmarking reproduzierbar und aussagekräftig gestalten 11
11.5 Spezialisierung, Parametrisierung und Grenzen untersuchen 12
11.6 Gestufter Arbeitsplan 12

A Quellcode-Ausschnitte **13**

1 Einführung

Diese Arbeit untersucht die Frage, welche booleschen Schaltkreise (als digitale, kombinatorische Schaltungen modelliert) SAT-erfüllbar sind, wie man mit nicht erfüllbaren Schaltungen umgeht und welche Folgerungen dies für Komplexitätstheorie und praktische Anwendungen hat. Die Untersuchung verbindet drei Artefakte des Autors: das Subgraph-Implementierungs-Ökosystem (csubgraph), die Reduktions-Implementierung subgraph-sat-solver und das Experimente-Framework subgraph-sat-solver-experiments. Die zentrale Perspektive ist: „SAT-erfüllbar bedeutet, die Formel (oder die Einbettungsfrage) kann durch einen SAT- oder Subgraph-Solver entschieden werden“ — wir behandeln insbesondere die Reduktion 3-SAT → Clique → Subgraph wie sie in subgraph-sat-solver implementiert ist.

Ziele:

- Formale Beweise der Korrektheit der Reduktionskette.
- Komplexitätstheoretische Konsequenzen (NP/Vollständigkeit, Implikationen für Klassen).
- Praktische Bedeutung: Einsatz, Grenzen, Entwurfsprinzipien.
- Analyse von ISCAS-Benchmarks: Kriterien zur Beurteilung ihrer SAT-Erfüllbarkeit, Empfehlungen und methodische Vorgehensweise.
- Verwendung der vorhandenen Implementierung (Codebeispiele aus subgraph-sat-solver) als Blaupause.

2 Begriffe und Modellierung

2.1 Boolesche Schaltkreise als Graphen und Formeln

Ein boolescher Schaltkreis (kombinatorisch) kann als gerichteter Acyclic Graph (DAG) modelliert werden: Gatterknoten (AND, OR, NOT, Input, Output) sind Knoten, Leitungen sind gerichtete Kanten. Durch Standard-Encodings kann ein Schaltkreis in eine konjunktive Normalform (CNF) über Variablen für die Signale transformiert werden; diese CNF ist erfüllbar genau dann, wenn es eine Eingangsbelegung gibt, die die gewünschte Eigenschaft erfüllt (z. B. eine spezifische Ausgabe).

Definitionen:

Definition 2.1. Eine CNF-Formel ist eine Konjunktion von Klauseln, jede Klausel eine Disjunktion von Literalen (Variable oder Negation). Eine Formel ist erfüllbar (SAT), wenn es eine Belegung aller Variablen gibt, die alle Klauseln erfüllt.

Durch Standard-Techniken (Tseitin-Transformation) lässt sich ein beliebiger Schaltkreis in eine CNF mit linearer Größe (in der Anzahl der Gatter) umgewandelt werden; dadurch ist die Frage nach Inputs, die eine gewünschte Ausgabe liefern, eine SAT-Frage.

2.2 Graph-Modelle: Clique und Subgraph

Als nächste Modellierungsebene nutzt die vorliegende Implementierung die klassische Reduktion von 3-SAT auf Clique und anschließend die Umformung in ein Subgraph-Problem. Ein Graph G mit einer k-Clique entspricht einer Auswahl von k paarweise kompatiblen Literalen (eine mögliche belegungsbasierte Wahl).

3 Formale Reduktionskette und Beweise

Wir geben vollständige Beweise der Korrektheit der Reduktionskette 3-SAT $\to$ Clique $\to$ Subgraph (Monomorphismus).

3.1 Satz: 3-SAT auf Clique (klassische Konstruktion)

Satz 3.1 (3-SAT auf Clique)**.** Sei $\varphi = C_1 \wedge \cdots \wedge C_m$ eine 3-SAT-Formel mit Klauseln C_i (je höchstens 3 Literale). Konstruiere den Graphen G_φ mit einem Knoten für jedes Literal-Vorkommen (i, p) (Klauselindex i, Position p in der Klausel) und Kanten zwischen (i, p) und (j, q) genau dann, wenn $i \neq j$ und die beiden Literale nicht komplementär sind. Dann gilt:

$$\varphi \text{ ist erfüllbar} \iff G_\varphi \text{ enthält eine Clique der Größe } m.$$

Beweis. ($\Rightarrow$) Sei α eine erfüllende Belegung von φ. Für jede Klausel C_i wähle ein Literal l_i in C_i welches unter α wahr ist. Betrachte die Menge $Q = \{(i, p_i) \mid i = 1, \ldots, m\}$ mit jeweils dem gewählten vorkommenden Literal. Da alle gewählten Literale wahr sind, kann es keine Paare l_i, l_j geben, die komplementär sind; außerdem sind die ausgewählten Knoten aus verschiedenen Klauseln. Nach Konstruktion sind daher alle Paare aus Q durch Kanten verbunden, also ist Q eine Clique der Größe m.

($\Leftarrow$) Sei Q eine Clique der Größe m in G_φ. Dann enthält Q genau einen Knoten aus jeder Klausel (weil Knoten aus derselben Klausel nicht verbunden sind). Markiere für jede Klausel C_i das Literal l_i das dem Knoten aus Q entspricht; aufgrund der Cliquen-Eigenschaft sind diese Literale paarweise nicht komplementär. Wähle die Belegung α so, dass jede Variable, die in irgendeinem l_i auftritt, so gesetzt wird, dass l_i wahr wird; für alle übrigen Variablen setze einen beliebigen Wert. Diese Belegung α erfüllt jede Klausel (jede Klausel hat mindestens das markierte wahr machende Literal) und ist konsistent, weil keine zwei markierten Literale komplementär sind. Somit ist φ erfüllbar. □

3.2 Korollar: Clique $\to$ Subgraph (K_k als Muster)

Korollar 3.1. Mit G_φ wie oben gilt:

$$G_\varphi \text{ enthält eine } m\text{-Clique} \iff K_m \sqsubseteq_{\text{mono}} G_\varphi,$$

wobei $\sqsubseteq_{\text{mono}}$ der Teilgraph-Monomorphismus-Begriff ist.

Beweis. Eine m-Clique ist gerade eine Kopie des vollständigen Graphen K_m als nicht-induzierter Teilgraph; damit besteht die Äquivalenz. □

3.3 Implizite Korrektheitsforderungen für die Subgraph-Oracle-Schnittstelle

Damit die Implementierung in subgraph-sat-solver die SAT-Korrektheit bewahrt, muss das Subgraph-Orakel eine echte Einbettung (injective Abbildung von $V(K_m)$ nach $V(G_\varphi)$, die Kanten erhält) zurückgeben oder zumindest korrekt mit „ja/neinäntworten. In der Praxis differenziert das Projekt zwischen einem heuristischen Signatur-orakel (Rotation/Signaturen) und einem exakten Entscheider (ExactSubgraph). Nur letzterer garantiert die vollständige Korrektheit der Reduktion; der heuristische Modus kann falsch-negative oder falsch-positive Antworten liefern. Die Implementierung (Adapter) berücksichtigt dies und bietet als Default den exakten Modus (siehe csubgraph: Solver-Adapter).

4 Komplexitätstheoretische Konsequenzen

4.1 NP-Vollständigkeit und Folgen

3-SAT ist NP-vollständig (Cook, 1971; Karp, 1972). Aus der korrekten polynomialen Reduktion 3-SAT $\leq_p$ Clique folgt, dass Clique NP-vollständig ist. Aus der Obiger Reduktion auf Subgraph-Instanzen (Erzeugung von G_φ und Ziel K_m) folgt, dass das allgemeine Subgraph-Isomorphismus-Problem (Entscheidung, ob H als Teilgraph in G enthalten ist) NP-vollständig ist, sobald $|H|$ als Teil der Eingabe variiert. Damit sind folgende Aussagen formell korrekt:

Satz 4.1. Entscheidungsprobleme, die allgemeine Schaltkreis-SAT-Anfragen abbilden, sind NP-vollständig.

Beweis. Jede Schaltkreis-SAT-Instanz kann in polynomialer Zeit in eine äquivalente 3-SAT-Instanz transformiert (Tseitin) werden. 3-SAT ist NP-vollständig, also ist auch Schaltkreis-SAT NP-hart; membership in NP ist offensichtlich (eine Zeugniskonstruktion: Eingangsbelegung). Damit NP-Vollständigkeit. □

4.2 Auswirkungen auf höhere Klassen

Eine praktische Folgerung: Ist ein praktisches Orakel in „gewöhnlicherSZeit (polynomiell) fähig, beliebige Subgraph-Instanzen zu entscheiden, so würde das P = NP implizieren. Da dies weithin als unwahrscheinlich erachtet wird, konzentriert sich Praxis auf Heuristiken, Spezialfälle, Parameterisierung (FPT) und approximative/Randomized-Verfahren. Für die hier betrachtete Kette bedeutet das: die Richtigkeit (vollständige, exakte Antwort) ist nur für instanzielle Größen handhabbar; für große Instanzen sind Heuristiken oder Parameter-Fixes (z. B. kleine Zielgröße k) erforderlich.

4.3 Parameterized Complexity

Betrachte das Problem „Existiert eine k-Clique in G?PParameterisiert nach k ist das Problem W[1]-vollständig; eine FPT-Lösung in k existiert nicht (sofern FPT≠W[1]). Allerdings existieren in der Praxis Algorithmen, die für kleine k sehr schnell arbeiten — die Clique-Implementierung in csubgraph benutzt z. B. k-Kernreduktion und Färbungs-Schranken.

5 Implementierung (subgraph-sat-solver)

In diesem Kapitel verknüpfen wir die theoretischen Aussagen mit der konkreten Implementierung. Aus Platzgründen zeigen wir zentrale Ausschnitte; vollständige Dateien liegen im Repository.

Listing 1: Auszug: CNF-Repräsentation und 3-SAT-Konversion (aus Cnf.h)

```
% Dateiquelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/Cnf.h
// ...
/**
 * Konvertiere beliebige k-SAT zu 3-SAT
 * Für jede Clause mit > 3 Literalen:
 * (a || b || c || d) -> (a || b || x1) && (!x1 || c || d)
 */
void to_3sat();
% ...
```

Die Funktionalität zur Vereinfachung (Duplikate, Tautologien, Unit-Propagation) ist wesentlich, weil sie die Größe der resultierenden Clique-/Subgraph-Instanz stark beeinflusst.

Listing 2: Auszug: Reduktionsklassen (aus Reduction.h)

```
% Dateiquelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/
    ↪ Reduction.h
// 3-SAT zu Clique Reduktion
// Knoten: Paare (lit, clause) für jeden Literal in jeder Clause
// Kanten: zwischen Knoten verschiedener Klauseln, sofern Literale kompatibel
class ThreeSATToCliqueReducer {
public:
    std::pair<Graph, int> reduce();
    bool validate() const;
private:
    const CNFFormula& formula_;
    bool literals_compatible(int lit1, int lit2) const;
};
```

Der Solver orchestriert die Schritte: Parsing DIMACS, Normalisierung, Reduktion, Serialisierung der Graphen als JSON, Aufruf des Subgraph-Binaries, Einlesen der Antwort und Belegungsextraktion. Relevanter Ablauf aus SubgraphSATSolver.cpp:

Listing 3: Auszug: Orchestrierung und Subgraph-Aufruf (aus SubgraphSATSolver.cpp)

```
% Dateiquelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/
    ↪ SubgraphSATSolver.cpp
// Schritt 1: Reduktion
SATToSubgraphReducer reducer(formula);
auto reduction_result = reducer.reduce();

// Schritt 2: Aufruf Subgraph-Binary
std::string subgraph_result_json;
if (!call_subgraph_binary(reduction_result.source_graph_json,
                          reduction_result.target_graph_json,
                          subgraph_result_json)) {
    // Fehlerbehandlung
```

```
}

// Schritt 3: Parse und Extraktion
auto mapping_opt = parse_subgraph_result(subgraph_result_json);
if (!mapping_opt) { result.status = UNSATISFIABLE; return result; }
SATAssignment full = SubgraphToSATExtractor::extract_assignment(...);
```

Die Implementierung prüft optional die gefundene Belegung gegen die Originalformel ($validate_result$).

6 SAT-Erfüllbarkeit für Schaltkreise (ISCAS-85/89)

Die ISCAS-Benchmarks sind standardisierte Schaltkreise (combinational und sequential). Um Aussagen über SAT-Erfüllbarkeit zu treffen, liefern wir hier ein genaues, reproduzierbares Verfahren (ohne lokale Ausführung; die Nutzer können dieses Verfahren mit subgraph-sat-solver und den Benchmarks nachmachen).

6.1 Vorgehen

1. Repräsentiere den Schaltkreis als CNF mittels Tseitin-Transformation. Für sequential circuits: wähle Unrolling-Länge t (bounded model checking).
2. Füge gewünschte Eigenschaft als Klauseln hinzu (z. B. Output = 1).
3. Vereinfachung: Unit-Propagation, Subsumption, Elimination definiter Literale.
4. Verwende subgraph-sat-solver (oder direkten SAT-Solver) zur Entscheidung.

6.2 Typische Befunde (generische Prinzipien)

Ohne konkrete Zusatzbedingungen sind viele ISCAS-Schaltungen *erfüllbar* für geeignete Eingangsbelegungen (weil sie einfach definierte logische Funktionen darstellen). Beispiele:

- Ein unveränderter combinational circuit, bei dem ein gewünschter Output-Bit Wert 1 sucht, ist normalerweise erfüllbar, es sei denn die Output-Funktion ist konstant 0.
- Ein Äquivalenzmiter (zwei implementierte Schaltungen sollen identisch sein): Die erzeugte CNF für „exists input such that outputs differ“ist SAT genau dann, wenn Implementierungen verschieden sind; UNSAT bedeutet funktionale Äquivalenz.
- Für sequential circuits hängt SAT von der Wahl von Unrolling-Länge t ab; manche Fehler sind nur bei größeren t sichtbar.

Wichtig: Ohne Ausführung kann man keine einzelnen ISCAS-Instanzen als SAT/UNSAT deklarieren. Stattdessen geben wir formale Kriterien:

Satz 6.1 (Erfüllbarkeitskriterium für kombinatorische ISCAS-Instanzen)**.** Sei C ein kombinatorischer Schaltkreis und sei o ein boolescher Ausdruck über seine Ausgänge. Nach Tseitin-Transformation ergibt sich Formel $\varphi_{C,o}$. Dann gilt:

$$\varphi_{C,o} \text{ ist unerfüllbar } \iff \forall x \in \{0,1\}^n : \; o(C(x)) = \text{false}.$$

Beweis: direkte Entsprechung der Semantik der Tseitin-Transformation.

6.3 Empfehlungen für die Praxis mit ISCAS

- Automatisiere Vorverarbeitung (Gate-Minimierung, CNF-Vereinfachung) — spart Platz in der Clique-Reduktion.
- Verwende exakte Subgraph-Entscheider nur für kleine bis mittlere Instanzen; für große Instanzen Heuristiken und SAT-Solver (CDCL) einsetzen.
- Für Äquivalenzprüfungen: zuerst BDD-basierte oder strukturbezogene Reduktionen (oder SAT mit miter) versuchen; nur wenn diese versagen, auf komplette Schleifen umschalten.

7 Design- und Entwurfsprinzipien für SAT-freundliche Schaltkreise

Aus Theorie und Erfahrung leiten sich robuste Entwurfsprinzipien ab:

7.1 Prinzipien

- Modularität: Kleine Module mit klaren Ein-/Ausgangs-Schnittstellen erzeugen lokalere CNF-Bausteine – das hilft SAT-Solver-Heuristiken.
- Vermeidung starker XOR-Cluster in kritischen Teilen: XOR-Netzwerke sind für viele SAT-Heuristiken schwieriger; falls unvermeidbar, alternative Repräsentationen/Unterformeln verwenden.
- Struktur beibehalten: Gatter-Struktur (Baumtiefe, Balancierung) beeinflusst Tseitin-Variablenanzahl und Clause-Interaktion.
- Parametrisierbarkeit: Entwurfsparameter, die als fixe Konstanten betrachtet werden können, reduzieren Variablenzahl und erleichtern SAT.

8 Beweise wichtiger Aussagen

Hier ergänzen wir die formalen Beweise für Korrektheit der Extraktion einer SAT-Belegung aus einer Subgraph-Einbettung (Subgraph ⇒ Assignment), wie sie in SubgraphToSATExtractor implementiert ist.

Satz 8.1 (Korrektheitsbelegung)**.** Sei f eine injektive Abbildung von $V(K_m)$ nach $V(G_\varphi)$, die jede Kante von K_m auf eine Kante in G_φ abbildet (Einbettung). Dann existiert eine Belegung α der Variablen der ursprünglichen Formel, so dass α alle Klauseln erfüllt, und diese Belegung ist rekonstruktiv aus f.

Beweis. Die Einbettung f wählt genau für jede Klausel C_i einen Repräsentantenknoten (i, p_i) in G_φ. Dieses Knotenpaar codiert das zu wählende Literal l_i. Wegen der Kantenbedingung können keine zwei gewählten Literale komplementär sein. Definiere α so, dass für jede Variable x die Werte so gesetzt werden, dass alle markierten Literale wahr werden (z. B. falls derselbe Variable sowohl positiv als auch negativ in markierten Literalen auftritt, wäre das ein Widerspruch; dies wird aber durch Kompatibilitätsbedingung ausgeschlossen). Für nicht referenzierte Variablen setze beliebig. Das resultierende α erfüllt jede Klausel, weil jede Klausel mindestens ihr markiertes Literal hat. Somit ist die Extraktion korrekt. □

9 Praktische Bewertung, Limitierungen und wirtschaftliche Relevanz

9.1 Leistung und Skalierbarkeit

Die Reduktion vergrößert die Instanz: Ein Knoten je Literal-Vorkommen, Adjazenzmatrix quadratisch in Literalen. Für große Schaltkreise mit vielen Vorkommen kann das unpraktikabel werden. Daher sind Optimierungen notwendig:

- Aggressive Vereinfachung vor Reduktion (Unit-Propagation, Tautologie-Elimination).
- Heuristische Partitionierung: teilinstanzielle Entscheidungsversuche, CEGAR-Ansätze.
- Parallele, verteilte Subgraph-Suche oder spezialisierte Clique-Algorithmen.

9.2 Wirtschaftliche Bedeutung

SAT-Entscheidungen treten in vielen industriellen Szenarien auf: Verifikation digitaler Schaltungen, Testvektor-Generierung, Fehlersuche, Konfigurationsmanagement, Logistik- und Planungsprobleme. Die Wahl einer Reduktionskette oder eines direkten SAT-Solvers hat Einfluss auf:

- Rechenkosten (Cloud/Cluster-Stunden).
- Entwicklungs- und Integrationskosten (Adapter, Formatkonvertierung).
- Antwortqualität (exakt vs. heuristisch).

Ökonomisch sinnvoll sind hybride Strategien: Erst schnelle Heuristiken und lokale Schnitte, bei Unschärfe exakte Verfahren für den kritischen Kern (z. B. via csubgraph ExactSubgraph).

10 Empfehlungen zur Nutzung

Die Implementierung in subgraph-sat-solver kann als reproduzierbares Experimentier-Backend genutzt werden. Konkrete Empfehlungen:

- Nutze die optionale Formel-Vereinfachung (`CNFFormula::simplify`, `to_3sat`) vor Reduktion.
- Im Adapter: setze `SUBGRAPH_MODE=exact` für verifizierbare Ergebnisse; nur für kleine bis mittlere Instanzen.
- Für große Schaltungen: bereite Partitionierungs- oder Abstraktionsstufen vor, bevor du Clique-Reduktion erzeugst.
- Protokolliere Laufzeiten je Phase (Reduktion, Subgraph-Aufruf, Extraktion) — SubgraphSATSolver führt das bereits in `SATSolverResult` aus.

11 Schluss

Die vorangegangenen Kapitel zeigen sowohl die prinzipielle Stärke als auch die praktischen Grenzen der untersuchten Reduktionskette. Für die weitere Entwicklung ist deshalb nicht allein entscheidend, ob sich eine Instanz in eine äquivalente Subgraph-Anfrage überführen lässt. Ebenso wichtig sind die Größe der erzeugten Graphen, die Zuverlässigkeit der Antwort, die Rekonstruierbarkeit eines Zeugnisses und die Reproduzierbarkeit der Messungen. Der folgende Ausblick ordnet mögliche Arbeiten entlang dieser Anforderungen.

11.1 Reduktion und Vorverarbeitung gezielt verbessern

Ein naheliegender erster Schritt ist eine systematische Vorverarbeitung der CNF, bevor der Clique-Graph konstruiert wird. Unit-Propagation, das Entfernen tautologischer Klauseln, das Zusammenfassen doppelter Literale und Klauseln sowie Subsumption können die Formel verkleinern, ohne ihre Erfüllbarkeit zu verändern. Solche Transformationen sollten als klar abgegrenzte Pipeline mit dokumentierten Vor- und Nachbedingungen umgesetzt werden. Für jede Stufe ist zu prüfen, ob sie lediglich die Formel vereinfacht oder ob zusätzlich Informationen gespeichert werden müssen, die später für die Rekonstruktion einer Belegung benötigt werden.

Die Größe des resultierenden Graphen sollte dabei als eigenes Entwurfsziel behandelt werden. Bei L Literal-Vorkommen kann die naive Konstruktion bis zu L Knoten und quadratisch viele mögliche Kanten umfassen. Es lohnt sich daher, die Graphstruktur zunächst über Kandidatenpaare und Klauselgruppen zu erzeugen, anstatt sofort eine dichte Adjazenzmatrix zu materialisieren. Eine sparse Repräsentation kann insbesondere bei Formeln mit geringer Dichte Speicher sparen. Vor jeder Implementierungsoptimierung sollten jedoch dieselben Instanzen und dieselben Korrektheitsprüfungen verwendet werden, damit Laufzeit- oder Speichergewinne nicht durch eine geänderte Semantik erkauft werden.

Ergänzend können Reduktionen untersucht werden, die besondere Eigenschaften der Eingabe ausnutzen. Dazu gehören eingeschränkte Klauselbreiten, begrenzte Variablenhäufigkeiten oder lokale Schaltungsstrukturen. Bei XOR-lastigen Schaltungen ist außerdem zu prüfen, ob eine Behandlung in einer geeigneteren Zwischenrepräsentation günstiger ist als eine unmittelbare Umwandlung in CNF. Entscheidend wäre jeweils ein präziser Vergleich: Welche äquivalente Entscheidungsfrage entsteht, wie groß ist sie, und welche Zusatzkosten entstehen für die Übersetzung des Ergebnisses zurück in eine Belegung?

11.2 Exakte und heuristische Verfahren sinnvoll kombinieren

Eine wichtige Weiterentwicklung ist eine hybride Solver-Architektur, in der die Subgraph-Reduktion nicht als einzig möglicher Lösungsweg behandelt wird. Ein direkter CDCL-SAT-Solver könnte als Referenz, Alternative oder Fallback dienen. Damit ließen sich dieselben CNF-Instanzen mit zwei methodisch unterschiedlichen Verfahren untersuchen: Die direkte SAT-Lösung arbeitet auf Klauseln, während die Reduktionskette die Struktur über Clique und Subgraph abbildet. Welche Strategie geeignet ist, hängt von der Instanz und dem Einsatzziel ab; es sollte daher nicht ohne Messdaten angenommen werden, dass eine Methode generell überlegen ist.

Eine solche Architektur sollte Ergebnisse samt Status eindeutig unterscheiden. Ein exaktes *SAT* oder *UNSAT* ist von einem Zeitlimit, einem Abbruch, einem nicht verfügbaren externen Programm und einer lediglich heuristischen Einschätzung zu trennen. Insbesondere darf ein nicht erfolgreich ausgeführter Subgraph-Aufruf nicht als negatives

Resultat interpretiert werden. Ein gemeinsames Ergebnisformat könnte neben dem Status auch die verwendete Methode, Laufzeit, Abbruchgrund und verfügbare Zeugnisse enthalten. So bliebe für Nutzende und für nachgelagerte Experimente nachvollziehbar, was tatsächlich entschieden wurde.

Eine mögliche Auswahlstrategie kann zunächst aus kostengünstigen Kennzahlen der normalisierten Formel und des erzeugten Graphen eine Verfahrensreihenfolge ableiten. Beispielsweise ließen sich Variablen- und Klauselzahl, Klauselbreiten, Literalhäufigkeiten, Knotenzahl und Kantendichte erfassen. Solche Kennzahlen sollten anfangs als beschreibende Merkmale dienen und nicht als unbelegte Garantie für eine bestimmte Laufzeit. Erst eine ausreichend breite und reproduzierbare Messreihe könnte zeigen, ob daraus belastbare Auswahlregeln oder ein trainierbares Vorhersagemodell entstehen.

11.3 Belege und Korrektheit durchgängig absichern

Für einen verlässlichen Einsatz sollte die Korrektheit nicht nur durch einen einzelnen Beweis der mathematischen Reduktion abgesichert werden. Jede Übersetzungsstufe benötigt Tests, die sowohl typische Fälle als auch Randfälle erfassen: leere Formeln, leere Klauseln, Einheitsklauseln, wiederholte oder komplementäre Literale, Variablen mit unterschiedlicher Polarität sowie triviale Zielgraphgrößen. Kleine Instanzen können zusätzlich exhaustiv ausgewertet werden. Für sie lässt sich die SAT-Antwort unabhängig durch Enumeration bestimmen und mit dem Resultat der Reduktionskette vergleichen.

Auch die Antwort des Subgraph-Solvers sollte, sofern sie eine Einbettung enthält, unabhängig validiert werden. Eine Prüfroutine kann kontrollieren, dass die Abbildung injektiv ist, die geforderten Kanten erhält und sich die ausgewählten Literal-Vorkommen konsistent zu einer Belegung zusammensetzen lassen. Anschließend ist die rekonstruierte Belegung direkt gegen die ursprüngliche Eingabeformel zu prüfen. Dieser letzte Schritt schützt insbesondere vor Fehlern in der Extraktion und in der Kommunikation zwischen Prozessen. Umgekehrt sollte ein UNSAT-Resultat nur dann als exaktes Ergebnis ausgegeben werden, wenn es aus einem verlässlichen vollständigen Entscheidungsverfahren stammt; eine fehlende gefundene Einbettung in einer Heuristik ist kein UNSAT-Beweis.

Langfristig wäre eine explizite Zertifikatsschnittstelle nützlich. Für SAT kann das Zertifikat eine Belegung sein, die ein unabhängiger Prüfer in linearer Zeit gegen die ursprüngliche Formel testet. Für UNSAT hängt ein prüfbares Zertifikat vom eingesetzten Verfahren ab; ein CDCL-Solver könnte beispielsweise einen geeigneten Beweis-Trace ausgeben, während ein Subgraph-Entscheider dafür eine eigene nachvollziehbare Beweisform bräuchte. Selbst wenn solche Zertifikate zunächst nicht implementiert werden, sollte die Architektur zwischen Antwort, Zeugnis und unabhängiger Prüfung unterscheiden.

11.4 Benchmarking reproduzierbar und aussagekräftig gestalten

Die Untersuchung der ISCAS-Benchmarks und weiterer Instanzfamilien bietet einen konkreten Rahmen für die Evaluation. Ein künftiges Experimentier-Framework sollte Eingabedateien, Vorverarbeitungsoptionen, Solver-Versionen, Konfigurationen und Zufalls-Seeds protokollieren. Zu den Messgrößen gehören nicht nur Gesamtlaufzeit und Speicherbedarf, sondern auch die Dauer der einzelnen Phasen: Parsing, Normalisierung, Reduktion, Serialisierung, externer Solver-Aufruf und Belegungsextraktion. Damit wird sichtbar, ob ein Engpass aus der Reduktion, der Graphrepräsentation oder dem eigentlichen Entscheider stammt.

Die Versuchsauswertung sollte gelöste Instanzen, Zeitüberschreitungen und technische Fehler getrennt berichten. Mittelwerte allein können bei stark schwankenden Laufzeiten irreführend sein; ergänzende Verteilungen, Medianwerte, Quantile und Angaben zu nicht gelösten Fällen liefern ein differenzierteres Bild. Für jeden Benchmark ist außerdem präzise anzugeben, welche Eigenschaft abgefragt wurde. Ein Schaltkreis ist nicht unabhängig von der Anfrage „erfüllbar“ oder „unerfüllbar“: Die konkrete CNF hängt davon ab, welche Ausgänge oder Abweichungen festgelegt werden. Bei sequentiellen Schaltungen müssen insbesondere die Unrolling-Tiefe und die daraus entstehende Bounded-Model-Checking-Frage dokumentiert werden.

Ein sorgfältig aufgebauter Vergleich sollte möglichst identische normalisierte Formeln an alle verglichenen Verfahren übergeben. So lassen sich Unterschiede im Solver-Verhalten von Unterschieden in der Übersetzung trennen. Neben aggregierten Kennzahlen sind auch einzelne repräsentative Fälle zu analysieren, etwa Instanzen, bei denen die Graphreduktion besonders stark wächst oder die Extraktion einen großen Teil der Gesamtlaufzeit ausmacht. Die Ergebnisse sollten schließlich so abgelegt werden, dass sie mit denselben Eingaben und Parametern erneut erzeugt werden können.

11.5 Spezialisierung, Parametrisierung und Grenzen untersuchen

Nicht jede Instanz muss mit einer allgemeinen Lösung behandelt werden. Für bestimmte Graph- oder Formelklassen können spezialisierte Algorithmen und Parameterisierungen deutliche Vorteile bieten. Die Clique-Reduktion legt insbesondere die Zielgröße k als möglichen Parameter nahe. Bei kleinen k können parameterisierte Verfahren interessant sein; für allgemeines k bleibt die bekannte Schwierigkeit des Clique-Problems jedoch bestehen. Eine praktische Untersuchung sollte deshalb die Parameterwerte und die Problemstruktur offenlegen, statt eine gute Leistung auf kleinen Zielgrößen auf allgemeine Instanzen zu übertragen.

Weitere mögliche Spezialisierungen betreffen Formeln mit begrenzter Variablenhäufigkeit, Schaltungen mit wiederverwendbaren Teilstrukturen oder Instanzen, die sich in unabhängige Teilprobleme zerlegen lassen. Hier könnten Dekomposition, Komponentenbildung oder inkrementelle Berechnung helfen. Bei wiederholten Anfragen an denselben Schaltkreis wäre zu erforschen, welche Teile der CNF oder Graphkonstruktion wiederverwendet werden können und welche Änderungen eine Neuberechnung erzwingen. Jede solche Optimierung verlangt jedoch eine klare Definition der Bedingungen, unter denen Wiederverwendung korrekt bleibt.

11.6 Gestufter Arbeitsplan

Für die Umsetzung bietet sich eine gestufte Reihenfolge an. Zunächst sollten Schnittstellen, Statuswerte und Korrektheitstests vereinheitlicht werden; dieser Schritt schafft eine belastbare Referenz für alle folgenden Änderungen. Danach kann die Vorverarbeitung mit nachvollziehbaren Statistiken erweitert und anhand kleiner sowie mittlerer Instanzen bewertet werden. Im dritten Schritt lässt sich die Integration eines direkten SAT-Verfahrens untersuchen und mit derselben Benchmark-Sammlung vergleichen. Erst wenn diese Messdaten vorliegen, sollten automatische Verfahrensauswahl und komplexere Vorhersagemodelle entwickelt werden.

Parallel dazu kann die Benchmark-Sammlung wachsen: Neben ISCAS-Instanzen sollten

synthetische Formeln enthalten sein, die gezielt einzelne Eigenschaften variieren, etwa Klauselbreite, Variablenhäufigkeit, Graphdichte oder Anteil von XOR-Struktur. So wird besser erkennbar, welche Eigenschaften tatsächlich mit der Leistung der Reduktionskette zusammenhängen. Die Ergebnisse jeder Entwicklungsstufe sollten mitsamt Quellcodeversion, Konfiguration und Rohdaten festgehalten werden. Das erleichtert nicht nur wissenschaftliche Replikation, sondern auch Regressionstests bei späteren Änderungen.

Insgesamt bleibt die Reduktionskette 3-SAT → Clique → Subgraph ein wertvolles formales und experimentelles Modell. Ihr Nutzen liegt nicht allein darin, einen weiteren Lösungsweg bereitzustellen, sondern auch darin, die Auswirkungen einer klaren mathematischen Transformation auf Repräsentation, Speicherbedarf, Laufzeit und Zeugnisextraktion konkret untersuchen zu können. Eine robuste Fortführung verbindet daher theoretische Korrektheit mit unabhängiger Ergebnisprüfung, sorgfältiger Messmethodik und einer ehrlichen Kennzeichnung der Grenzen heuristischer Verfahren. Auf dieser Grundlage kann aus der bestehenden Implementierung schrittweise ein reproduzierbares Vergleichs- und Forschungswerkzeug werden.

A Quellcode-Ausschnitte

Siehe im Text die eingebetteten Ausschnitte aus den Implementierungsdateien:

- Cnf.h — CNF-Parsing, 3-SAT-Konversion (Quelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/Cnf.h)
- Reduction.h — Reduktionsklassen (Quelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/Reduction.h)
- SubgraphSATSolver.cpp — Orchestrierung und Subgraph-Aufruf (Quelle: https://github.com/naphez86/subgraph-sat-solver/blob/main/SubgraphSATSolver.cpp)

Literatur

[1] S. A. Cook. The complexity of theorem-proving procedures. Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, 1971.

[2] R. M. Karp. Reducibility among combinatorial problems. 1972.

[3] M. Davis and H. Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960.

[4] M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.

[5] G. S. Tseitin. On the complexity of derivation in propositional calculus. In *Studies in Constructive Mathematics and Mathematical Logic*, Part II, pages 115–125, 1968.

[6] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[7] C. H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.

[8] R. G. Downey and M. R. Fellows. *Fundamentals of Parameterized Complexity*. Springer, 2013.

[9] J. R. Ullmann. An algorithm for subgraph isomorphism. *Journal of the ACM*, 23(1):31–42, 1976.

[10] F. Brglez and H. Fujiwara. A neutral netlist of 10 combinational benchmark circuits and a target translator in FORTRAN. In *Proceedings of the IEEE International Symposium on Circuits and Systems*, pages 663–698, 1985.

[11] F. Brglez, D. Bryan, and K. Kozminski. Combinational profiles of sequential benchmark circuits. In *Proceedings of the IEEE International Symposium on Circuits and Systems*, 1989.

[12] A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors. *Handbook of Satisfiability*. IOS Press, 2009.

[13] N. Een and N. Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, pages 502–518, 2003.

[14] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference*, pages 530–535, 2001.

[15] J. P. Marques-Silva and K. A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[16] N. Een and A. Biere. Effective preprocessing in SAT through variable and clause elimination. In *Theory and Applications of Satisfiability Testing (SAT 2005)*, pages 61–75, 2005.

[17] J. P. Marques-Silva. Practical applications of Boolean satisfiability. In A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors, *Handbook of Satisfiability*. IOS Press, 2009.

[18] E. Tomita, Y. Sutani, T. Higashi, S. Takahashi, and M. Wakatsuki. A simple and faster branch-and-bound algorithm for finding a maximum clique. In *WALCOM: Algorithms and Computation*, pages 191–203, 2010.

[19] P. M. Pardalos and J. Xue. The maximum clique problem. *Journal of Global Optimization*, 4:301–328, 1994.